Bitácora de la práctica 7: Herencia

Nombre: Hernández Nieves Emily
Número de cuenta: 426058246
Grupo: 6

En esta práctica trabajé con el concepto de herencia en Java, creando dos jerarquías de clases, una relacionada con animales y otra con personajes de videojuegos. El objetivo aquí fue comprender cómo una clase puede heredar atributos y métodos de otra, también utilizar constructores, getters y la sobrescritura de métodos.
En la primera parte desarrollé las clases Animal, Perro, PerroGuia y Gato. Para ello utilicé extends para establecer la relación entre las clases y super() para llamar al constructor de la clase padre. También utilicé @Override para modificar el comportamiento de métodos como sonido() y toString(). Al ejecutar MainAnimales, obtuve la salida esperada, incluyendo la validación de la edad negativa de Lomito, que se guardó como cero.
En la segunda parte trabajé con las clases Personaje, Caballero, Paladin y Mago. En esta sección apliqué lo aprendido anteriormente, procurando que cada clase tuviera sus propios atributos privados y sus respectivos métodos de acceso. También implementé la sobrescritura de atacar() y toString(), utilizando los métodos de la clase padre cuando era necesario. Al ejecutar MainPersonajes, los resultados coincidieron con los indicados en la guía, incluyendo el cálculo del ataque de Lunara, que fue de 42.

Para comprobar el orden en que se ejecutan los constructores, agregué temporalmente mensajes al final de los constructores de Personaje, Caballero y Paladin. Al ejecutar el programa, observé que primero se ejecuta el constructor de la clase padre y después el de la clase hija.
Esto se pudo observar al crear a Brakus, ya que primero apareció el mensaje de Personaje y después el de Caballero. En el caso de Lunara aparecieron tres mensajes: primero Personaje listo, después Caballero listo y finalmente Paladin listo. Esto sucede porque Paladin hereda de Caballero y Caballero, a su vez, hereda de Personaje, entonces, los constructores se ejecutan desde la clase más general hasta llegar a la más específica.
Al terminar la comprobación, retiré los mensajes temporales para conservar la salida original solicitada en la práctica

Esta práctica me ayudó a comprender cómo funciona la herencia en Java y cómo se pueden reutilizar métodos y atributos sin tener que repetir tanto código en cada clase. También reforcé el uso de super(), @Override y los constructores en cadena. Siempre es útil la parte de comprobar los resultados ejecutando los programas, porque así pude observar de manera más clara cómo se relacionan las clases y en qué orden se ejecutan sus constructores.

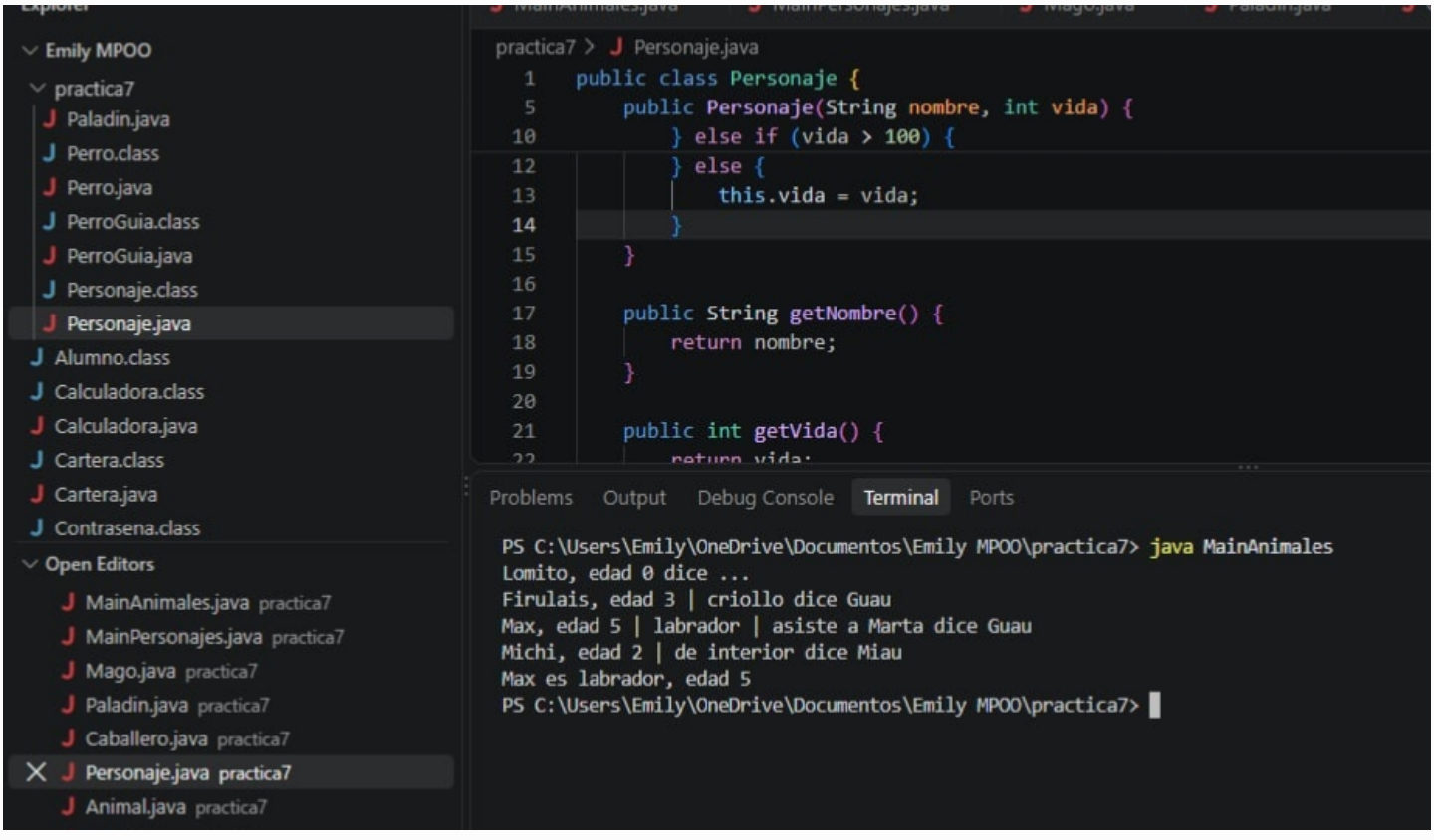


***Figura 1**. Ejecución de Animales.java.*

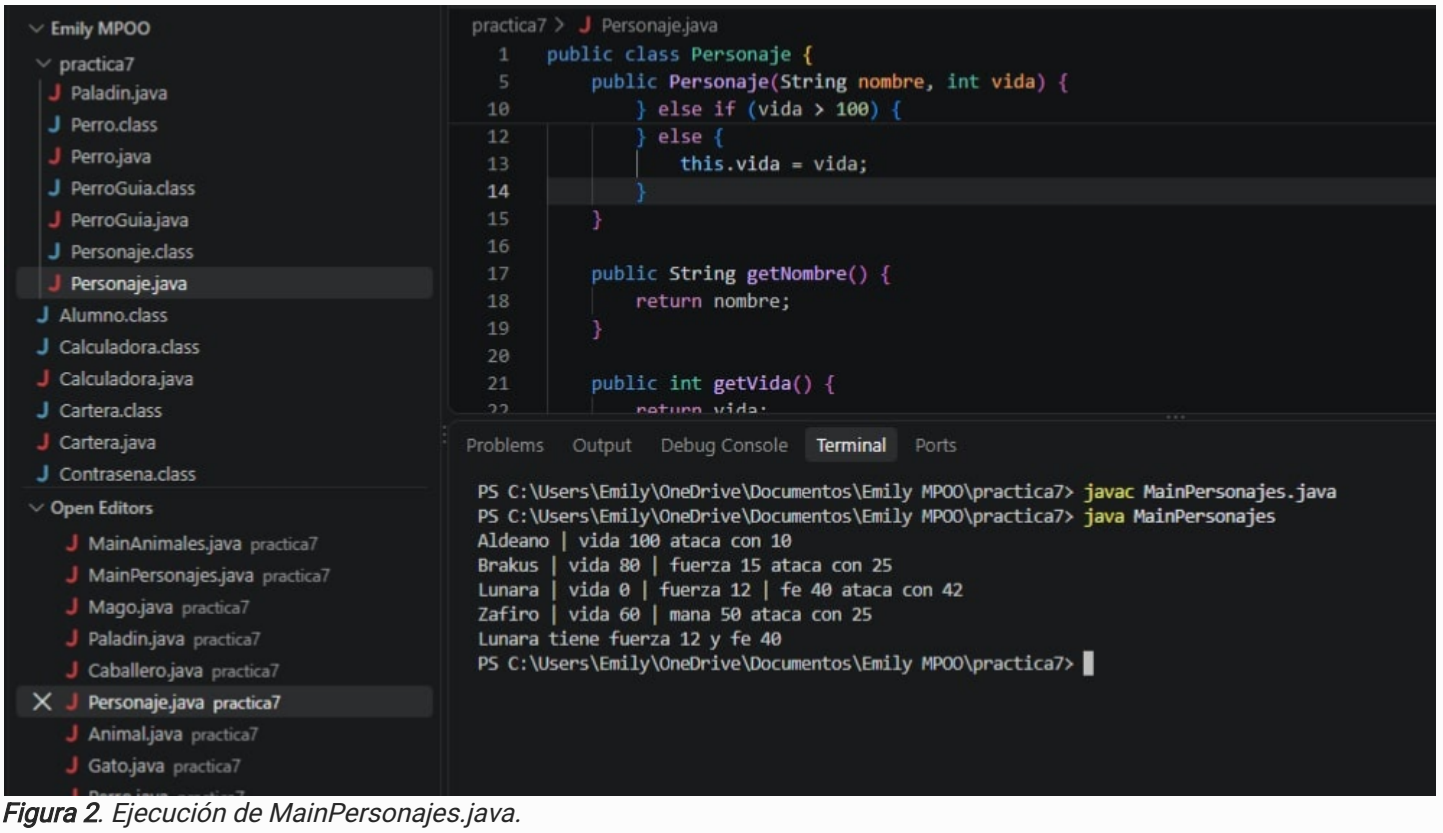


***Figura 2**. Ejecución de MainPersonajes.java.*

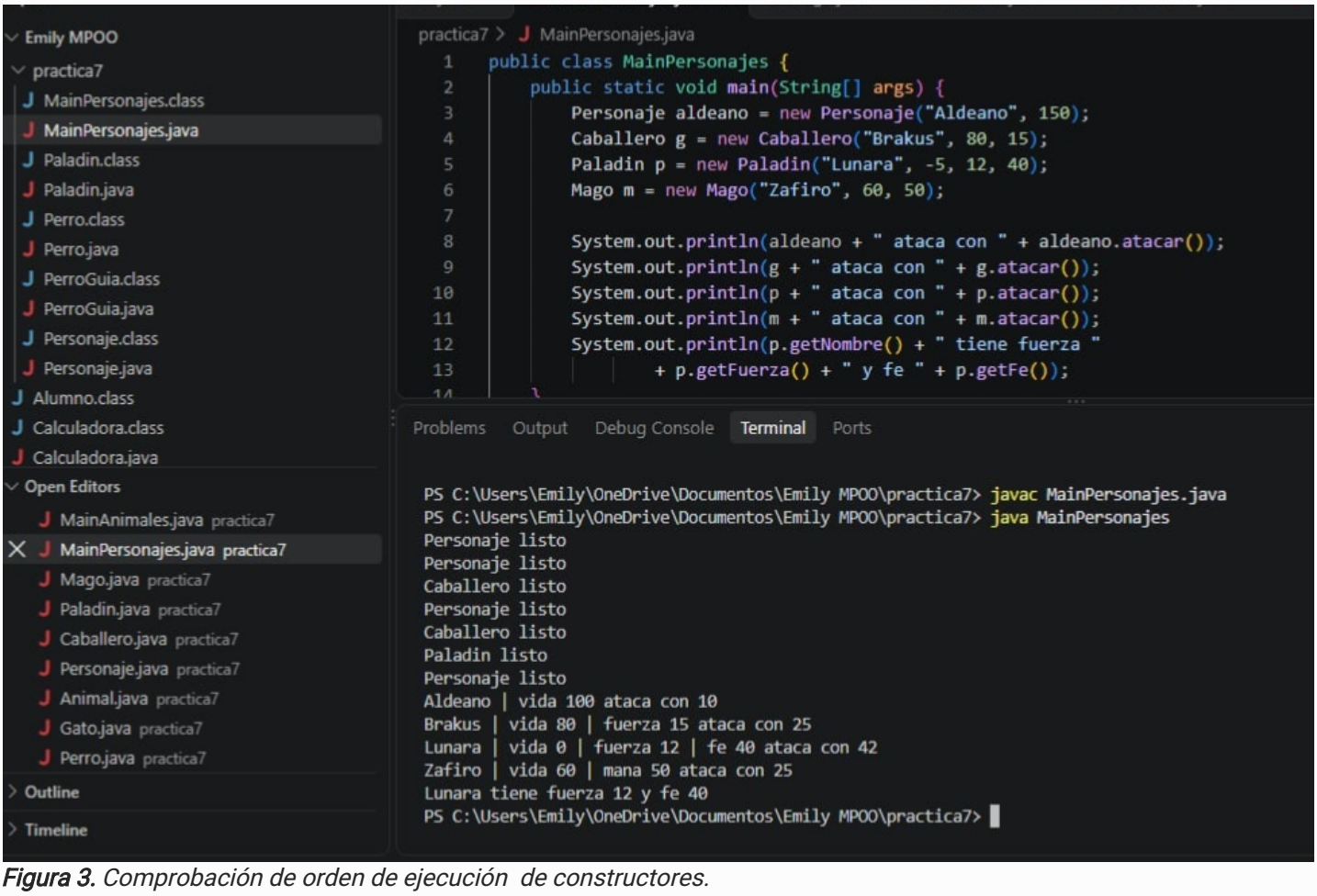


Figura 3. *Comprobación de orden de ejecución de constructores.*